

Homework 3: STLC

The appendix gives the formalization of STLC and its extensions with products and sums.

1. Consider the following term M :

$$\text{case } (\text{right } \langle \lambda x : \mathbf{T}. x, \lambda x : \mathbf{T}. x \rangle) \text{ do } (\lambda f : \mathbf{T} \rightarrow \mathbf{T}. f) \\ (\lambda p : (\mathbf{T} \rightarrow \mathbf{T}) \times (\mathbf{T} \rightarrow \mathbf{T}). \lambda x : \mathbf{T}. (\text{proj1 } p)((\text{proj2 } p) x))$$

Find the type τ of the term M above and give a complete typing derivation of $\bullet \vdash M : \tau$. Besides, reduce M to a normal form. Please do not skip steps.

2. **Pair-binding let.**

In this problem we add “pair-binding let” to STLC with products.

Syntax (types and values are unchanged):

$$M ::= \dots \mid \text{let } \langle x, y \rangle = M \text{ in } N$$

In $\text{let } \langle x, y \rangle = M \text{ in } N$, x and y scope over N but not M , and x and y are distinct variable names.

The free-variable definition is extended by

$$fv(\text{let } \langle x, y \rangle = M \text{ in } N) \stackrel{\text{def}}{=} fv(M) \cup (fv(N) - \{x, y\}).$$

New typing rule:

$$\frac{\Gamma \vdash M : \sigma \times \tau \quad \Gamma, x : \sigma, y : \tau \vdash N : \rho}{\Gamma \vdash \text{let } \langle x, y \rangle = M \text{ in } N : \rho} \text{ (LET-PAIR)}$$

New reduction rules (here v_1 and v_2 are values):

$$\frac{}{\text{let } \langle x, y \rangle = \langle v_1, v_2 \rangle \text{ in } N \longrightarrow N[v_1/x, v_2/y]} \text{ (LET-PAIR-V)}$$

$$\frac{M \longrightarrow M'}{\text{let } \langle x, y \rangle = M \text{ in } N \longrightarrow \text{let } \langle x, y \rangle = M' \text{ in } N} \text{ (LET-M)}$$

Here $N[U/x, V/y]$ is simultaneous substitution, defined by

$$N[U/x, V/y] \stackrel{\text{def}}{=} N[a/x][b/y][U/a][V/b], \quad \text{where } a, b \text{ are fresh}$$

In the above definition of simultaneous substitution, all substitutions on the right are ordinary substitutions that avoid name capture. Ordinary substitution extends to “pair-binding let” as follows:

$$\begin{aligned}
(\text{let } \langle x, y \rangle = M \text{ in } N)[U/z] &\stackrel{\text{def}}{=} \text{let } \langle x, y \rangle = M[U/z] \text{ in } N \\
&\text{if } z \in \{x, y\} \\
(\text{let } \langle x, y \rangle = M \text{ in } N)[U/z] &\stackrel{\text{def}}{=} \text{let } \langle x, y \rangle = M[U/z] \text{ in } N[U/z] \\
&\text{if } z \notin \{x, y\} \text{ and } \{x, y\} \cap \text{fv}(U) = \emptyset \\
(\text{let } \langle x, y \rangle = M \text{ in } N)[U/z] &\stackrel{\text{def}}{=} \text{let } \langle a, b \rangle = M[U/z] \text{ in } N[a/x][b/y][U/z] \\
&\text{if } z \notin \{x, y\} \text{ and } \{x, y\} \cap \text{fv}(U) \neq \emptyset \text{ and } a, b \text{ are fresh}
\end{aligned}$$

(a) Consider the following term M :

$$\begin{aligned}
&\text{let } \langle f, g \rangle = \\
&\quad \langle (\lambda h : \mathbf{T} \rightarrow \mathbf{T}. h)(\lambda a : \mathbf{T}. a), \lambda b : \mathbf{T}. b \rangle \\
&\text{in } (\lambda k : \mathbf{T} \rightarrow \mathbf{T}. \lambda z : \mathbf{T}. f(k z)) g
\end{aligned}$$

- i. Find the type τ of the term M and give a complete typing derivation of $\bullet \vdash M : \tau$.
- ii. Reduce M to a normal form. Please do not skip steps.

(b) Consider the following term U :

$$\begin{aligned}
&\text{let } \langle f, g \rangle = \langle \lambda h : \mathbf{T} \rightarrow \mathbf{T}. h, \lambda x : \mathbf{T}. x \rangle \\
&\text{in } \text{let } \langle k, p \rangle = \langle f, \langle g, f \rangle \rangle \\
&\quad \text{in } \langle k(\text{proj1 } p), \text{proj2 } p \rangle
\end{aligned}$$

- i. Find the type τ of the term U and give a complete typing derivation of $\bullet \vdash U : \tau$.
- ii. Reduce U to a normal form. Please do not skip steps.

(c) **Preservation and progress after changing rules.** Each variant changes the *original* language independently. All typing rules, values, and other reduction rules remain unchanged.

Variant 1. Replace LET-PAIR-V by

$$\frac{}{\text{let } \langle x, y \rangle = \langle M_1, M_2 \rangle \text{ in } N \longrightarrow N[M_1/x, M_2/y]} \text{ (LET-PAIR-ANY)}$$

Variant 2. Add

$$\frac{N \longrightarrow N'}{\text{let } \langle x, y \rangle = M \text{ in } N \longrightarrow \text{let } \langle x, y \rangle = M \text{ in } N'} \text{ (LET-N)}$$

Variant 3. Remove LET-M.

Variant 4. Replace LET-PAIR-V by

$$\overline{\text{let } \langle x, y \rangle = \langle v_1, v_2 \rangle \text{ in } N \longrightarrow N[v_2/x, v_1/y]} \quad (\text{LET-PAIR-SWAP})$$

Variant 5. Replace LET-PAIR-V by

$$\overline{\text{let } \langle x, y \rangle = \langle v_1, v_2 \rangle \text{ in } N \longrightarrow N[v_1/x][v_2/y]} \quad (\text{LET-PAIR-SEQ})$$

Fill the table with *holds* or *fails*, using preservation and progress as stated in the appendix. For each failed property, give a counterexample and explain why it violates that property. No explanation is required for a *holds* entry.

Variant	Preservation	Progress
1		
2		
3		
4		
5		

Appendix

A The Simply-Typed λ -Calculus

Syntax (here T denotes the base type):

$$\begin{aligned} (\text{Types}) \quad \tau &::= \mathsf{T} \mid \tau \rightarrow \tau \\ (\text{Terms}) \quad M &::= x \mid \lambda x : \tau. M \mid M N \\ (\text{Values}) \quad v &::= \lambda x : \tau. M \\ (\text{Contexts}) \quad \Gamma &::= \bullet \mid \Gamma, x : \tau \end{aligned}$$

Here Γ is a finite mapping from variable names to types. In a context extension $\Gamma, x : \tau$, we require $x \notin \text{dom}(\Gamma)$. Typing is considered up to α -equivalence, so bound variables may be renamed to satisfy this condition.

Free variables:

$$\begin{aligned} fv(x) &\stackrel{\text{def}}{=} \{x\}, \\ fv(M N) &\stackrel{\text{def}}{=} fv(M) \cup fv(N), \\ fv(\lambda x : \tau. M) &\stackrel{\text{def}}{=} fv(M) - \{x\}. \end{aligned}$$

Substitution (here x and y are distinct variable names):

$$\begin{aligned} x[N/x] &\stackrel{\text{def}}{=} N, \\ y[N/x] &\stackrel{\text{def}}{=} y, \\ (M_1 M_2)[N/x] &\stackrel{\text{def}}{=} (M_1[N/x]) (M_2[N/x]), \\ (\lambda x : \tau. M)[N/x] &\stackrel{\text{def}}{=} \lambda x : \tau. M, \\ (\lambda y : \tau. M)[N/x] &\stackrel{\text{def}}{=} \lambda y : \tau. (M[N/x]) \quad \text{if } y \notin fv(N), \\ (\lambda y : \tau. M)[N/x] &\stackrel{\text{def}}{=} \lambda z : \tau. (M[z/y])[N/x] \quad \text{if } y \in fv(N) \text{ and } z \text{ fresh.} \end{aligned}$$

Reduction rules:

$$\begin{aligned} \frac{}{(\lambda x : \tau. M) N \longrightarrow M[N/x]} \quad & \frac{M \longrightarrow M'}{\lambda x : \tau. M \longrightarrow \lambda x : \tau. M'} \\ \frac{M \longrightarrow M'}{M N \longrightarrow M' N} \quad & \frac{N \longrightarrow N'}{M N \longrightarrow M N'} \end{aligned}$$

Multistep reduction ($k \geq 0$):

$$\begin{aligned} M \longrightarrow^0 M' &\quad \text{iff } M = M', \\ M \longrightarrow^{k+1} M' &\quad \text{iff } \exists M''. M \longrightarrow M'' \wedge M'' \longrightarrow^k M', \\ M \longrightarrow^* M' &\quad \text{iff } \exists k \geq 0. M \longrightarrow^k M'. \end{aligned}$$

Normal form: a term M for which no M' satisfies $M \longrightarrow M'$.

Closed term: a term M with $fv(M) = \emptyset$.

Typing rules:

$$\begin{array}{c} \frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \text{ (VAR)} \qquad \frac{\Gamma, x : \tau \vdash M : \tau'}{\Gamma \vdash (\lambda x : \tau. M) : \tau \rightarrow \tau'} \text{ (ABS)} \\[10pt] \frac{\Gamma \vdash M : \tau \rightarrow \tau' \quad \Gamma \vdash N : \tau}{\Gamma \vdash M N : \tau'} \text{ (APP)} \end{array}$$

Preservation:

For any M, M' and τ , if $\bullet \vdash M : \tau$ and $M \longrightarrow M'$, then $\bullet \vdash M' : \tau$.

Progress:

For any M and τ , if $\bullet \vdash M : \tau$, then either $M \in \text{Values}$ or $\exists M'. M \longrightarrow M'$.

A.1 Products

Syntax:

$$\begin{array}{ll} \text{(Types)} & \tau ::= \dots \mid \tau \times \tau \\ \text{(Terms)} & M ::= \dots \mid \langle M, M \rangle \mid \text{proj1 } M \mid \text{proj2 } M \\ \text{(Values)} & v ::= \dots \mid \langle v, v \rangle \end{array}$$

Free variables:

$$\begin{aligned} fv(\langle M, N \rangle) &\stackrel{\text{def}}{=} fv(M) \cup fv(N), \\ fv(\text{proj1 } M) &\stackrel{\text{def}}{=} fv(M), \quad fv(\text{proj2 } M) \stackrel{\text{def}}{=} fv(M). \end{aligned}$$

Typing rules:

$$\begin{array}{c} \frac{\Gamma \vdash M : \sigma \quad \Gamma \vdash N : \tau}{\Gamma \vdash \langle M, N \rangle : \sigma \times \tau} \text{ (PAIR)} \\[10pt] \frac{\Gamma \vdash M : \sigma \times \tau}{\Gamma \vdash \text{proj1 } M : \sigma} \text{ (PROJ1)} \quad \frac{\Gamma \vdash M : \sigma \times \tau}{\Gamma \vdash \text{proj2 } M : \tau} \text{ (PROJ2)} \end{array}$$

Reduction rules:

$$\begin{array}{c} \overline{\text{proj1 } \langle M, N \rangle \longrightarrow M} \quad \overline{\text{proj2 } \langle M, N \rangle \longrightarrow N} \\[10pt] \frac{M \longrightarrow M'}{\langle M, N \rangle \longrightarrow \langle M', N \rangle} \quad \frac{N \longrightarrow N'}{\langle M, N \rangle \longrightarrow \langle M, N' \rangle} \\[10pt] \frac{M \longrightarrow M'}{\text{proj1 } M \longrightarrow \text{proj1 } M'} \quad \frac{M \longrightarrow M'}{\text{proj2 } M \longrightarrow \text{proj2 } M'} \end{array}$$

A.2 Sums

Syntax:

$$\begin{array}{ll}
\text{(Types)} & \tau ::= \dots \mid \tau + \tau \\
\text{(Terms)} & M ::= \dots \mid \text{left } M \mid \text{right } M \mid \text{case } M \text{ do } M_1 M_2 \\
\text{(Values)} & v ::= \dots \mid \text{left } v \mid \text{right } v
\end{array}$$

Free variables:

$$\begin{aligned}
fv(\text{left } M) &\stackrel{\text{def}}{=} fv(M), & fv(\text{right } M) &\stackrel{\text{def}}{=} fv(M), \\
fv(\text{case } M \text{ do } M_1 M_2) &\stackrel{\text{def}}{=} fv(M) \cup fv(M_1) \cup fv(M_2).
\end{aligned}$$

Typing rules:

$$\begin{array}{c}
\frac{\Gamma \vdash M : \sigma}{\Gamma \vdash \text{left } M : \sigma + \tau} \text{ (LEFT)} \quad \frac{\Gamma \vdash M : \tau}{\Gamma \vdash \text{right } M : \sigma + \tau} \text{ (RIGHT)} \\
\\
\frac{\Gamma \vdash M : \sigma + \tau \quad \Gamma \vdash M_1 : \sigma \rightarrow \rho \quad \Gamma \vdash M_2 : \tau \rightarrow \rho}{\Gamma \vdash \text{case } M \text{ do } M_1 M_2 : \rho} \text{ (CASE)}
\end{array}$$

Reduction rules:

$$\begin{array}{c}
\frac{}{\text{case } (\text{left } M) \text{ do } M_1 M_2 \longrightarrow M_1 M} \\
\\
\frac{}{\text{case } (\text{right } M) \text{ do } M_1 M_2 \longrightarrow M_2 M} \\
\\
\frac{M \longrightarrow M'}{\text{left } M \longrightarrow \text{left } M'} \quad \frac{M \longrightarrow M'}{\text{right } M \longrightarrow \text{right } M'} \\
\\
\frac{M \longrightarrow M'}{\text{case } M \text{ do } M_1 M_2 \longrightarrow \text{case } M' \text{ do } M_1 M_2} \\
\\
\frac{M_1 \longrightarrow M'_1}{\text{case } M \text{ do } M_1 M_2 \longrightarrow \text{case } M \text{ do } M'_1 M_2} \\
\\
\frac{M_2 \longrightarrow M'_2}{\text{case } M \text{ do } M_1 M_2 \longrightarrow \text{case } M \text{ do } M_1 M'_2}
\end{array}$$

Substitution extends componentwise to products and sums.
 Preservation and progress also hold with products and sums.